
HPC Clusters and Programming

Milind Chabbi

Nikhil Hegde

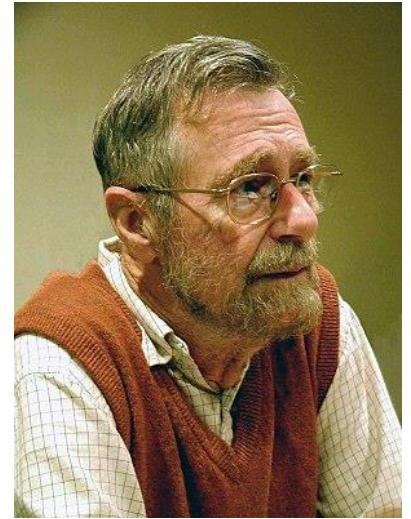
Department of Computer Science and Engineering

IIT Dharwad

*"as long as there were no machines,
programming was no problem at all;*

*when we had a few weak computers,
programming became a mild problem, and*

*now we have gigantic computers,
programming has become an equally
gigantic problem."*



pic: https://en.wikipedia.org/wiki/Edsger_W._Dijkstra

- Edgar Dijkstra, 1972 Turing Award Lecture

A Gigantic Computer

- System 360 / Model 91



Source: https://www.ibm.com/ibm/history/exhibits/mainframe/mainframe_PP2091.html

Topics for Today

- **Decomposition techniques**
- **Characteristics of tasks and interactions**
- **Mapping techniques for load balancing**

What is a Cluster ?

- Gigantic computer
 - from interconnecting several smaller computers



VIRGO Super Cluster, IIT Madras. Source: <https://cc.iitm.ac.in/node/184>

What is a Cluster ?

- Gigantic computer
 - from interconnecting several smaller computers
- Compute power in the order of 10^{15} floating point operations per second (Peta* FLOPS)
 - Your i7-based personal computer – few Giga FLOPs (10^9)

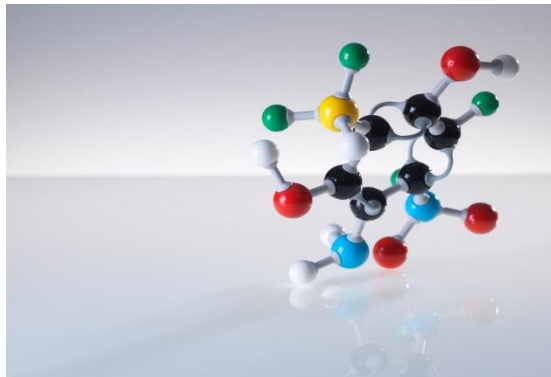
What is a Cluster ?

- Gigantic computer
 - from interconnecting several smaller computers
- Compute power in the order of 10^{15} floating point operations per second (Peta* FLOPS)
 - Your i7-based personal computer – few Giga FLOPs (10^9)
- E.g.
 - Chandra (IIT Palakkad), Virgo (IIT Madras), AnantGanak (IIT Dharwad) etc.

Why Clusters?



Financial Analysis



Genomics



Design Simulation



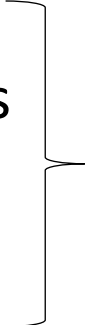
Oil Exploration



Weather Prediction

pic source: stock images

Terminology - Cluster Elements

- Processing Element
 - Core, CPU, Node, GPU, Virtual CPUs
 - Storage
 - Interconnect
- 
- Hardware**
- Partition
 - Job and Job Scheduler
 - Operating System (OS), Software Development Tools, Applications
- 
- Software**
- *Infrastructure* – power, cooling,

Processing Element

- Node

- Standalone computer
- Comprised of multiple CPUs/Processors/Cores, memory, network interfaces.



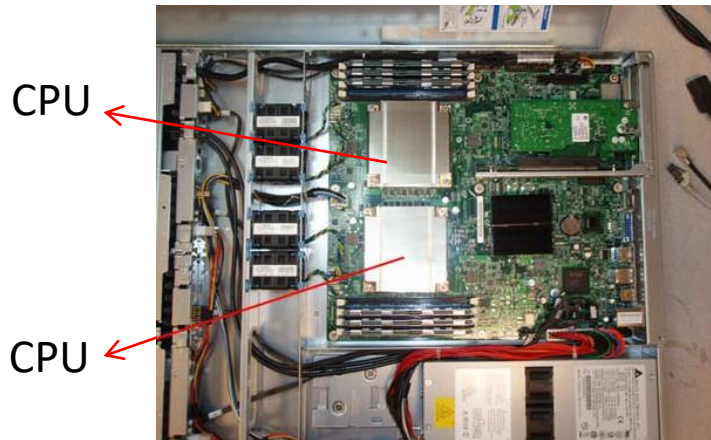
Each Green dot is a Node

- Master / Log-in node: is what end user interacts with (think: operator's console)

Processing Element

- CPU/Processor and Socket

- No consensus on terminology. Some vendors call multi-core CPUs as sockets.



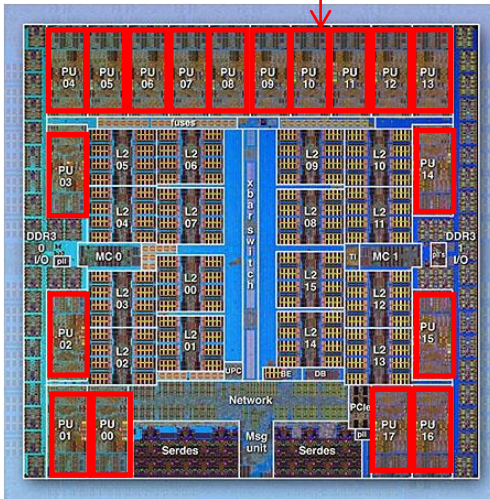
source: Blaise Barney, Introduction to Parallel Computing,
<https://hpc.llnl.gov/training/tutorials/introduction-parallel-computing-tutorial>

- Socket can also be a place to plug a CPU. E.g. dual-socket motherboard in pic.

Processing Element

- Core

- Each PU shown is a core. Pic: IBM BG/Q with 18 Cores

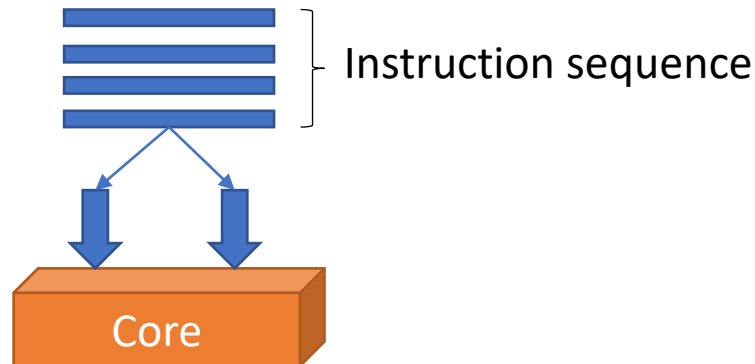


source: Blaise Barney, Introduction to Parallel Computing,
<https://hpc.llnl.gov/training/tutorials/introduction-parallel-computing-tutorial>

- In the past, each CPU (with just one core) was a single execution unit. Now, each core is an independent execution unit.

Processing Element

- Thread (hardware)
 - Pathway for flow of instruction within a core
 - When exposed to the OS, the OS gives an illusion to the programmer that multiple cores exist (“HyperThreading”)



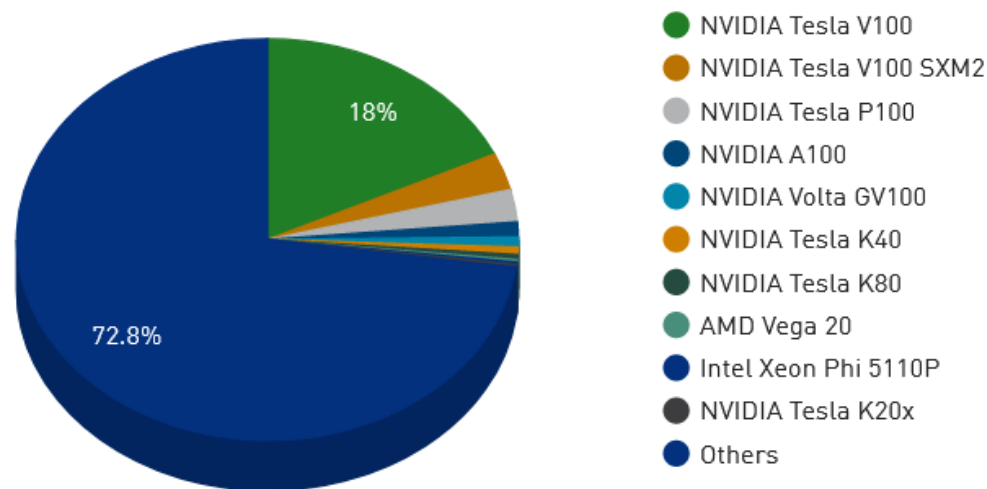
Processing Element

- Virtual CPUs
 - A term that you often get to hear when working with 'Cloud' hosted systems
 - Virtual Machine (VM) assigned to a single physical core
 - A VM is an abstraction/emulation of a computer

Processing Element

- GPU (Graphics Processing Unit) – Add-on devices
 - Traditionally: accelerate image creation
 - Now: GPGPUs for large-scale modeling, genetic programming

Accelerator/Co-Processor System Share



source:top500.org

Cluster Elements

- Interconnect
 - Nvidia-Mellanox Switch

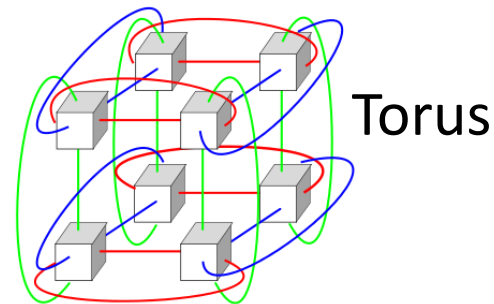


source: <https://www.nvidia.com/en-in/networking/infiniband-switching/>

- Cabling (determines topology)



<https://www.rcac.purdue.edu/training/clusters101/>



source: https://en.wikipedia.org/wiki/Torus_interconnect

Cluster Elements

- Storage

- \$HOME

- Landing directory when you log in to the master node

- \$SCRATCH and/or Parallel File System

- A fast memory where you should keep all the data needed for executing the task
 - E.g. Lustre, BeeGFS, etc.

- (Optionally) storage may be available on each compute node e.g. /tmp

- (Optionally) archival storage (e.g. LTO-6, Storage Server (e.g. IBM DS3512))

Cluster Elements - Software

- Job

- A **task** performed by the cluster
- Represented by a set of commands to the cluster, captured in a **script**, to precisely tell how to execute the task
- Usually, the set of commands do not require your intervention i.e. non-interactive
 - You issue the commands (read: “submit a job”) and go for coffee..

Cluster Elements - Software

- **Batch System** - Job Scheduler and Resource manager
 1. Provide a user interface to submit, monitor, and run jobs
 2. Manage the computational resources mentioned previously
 3. Implement the usage policies set by HPC Admin
- E.g. SLURM (Simple Linux Utility for Resource Management), PBS (Portable Batch System) – Torque, Moab.

Cluster Elements - Software

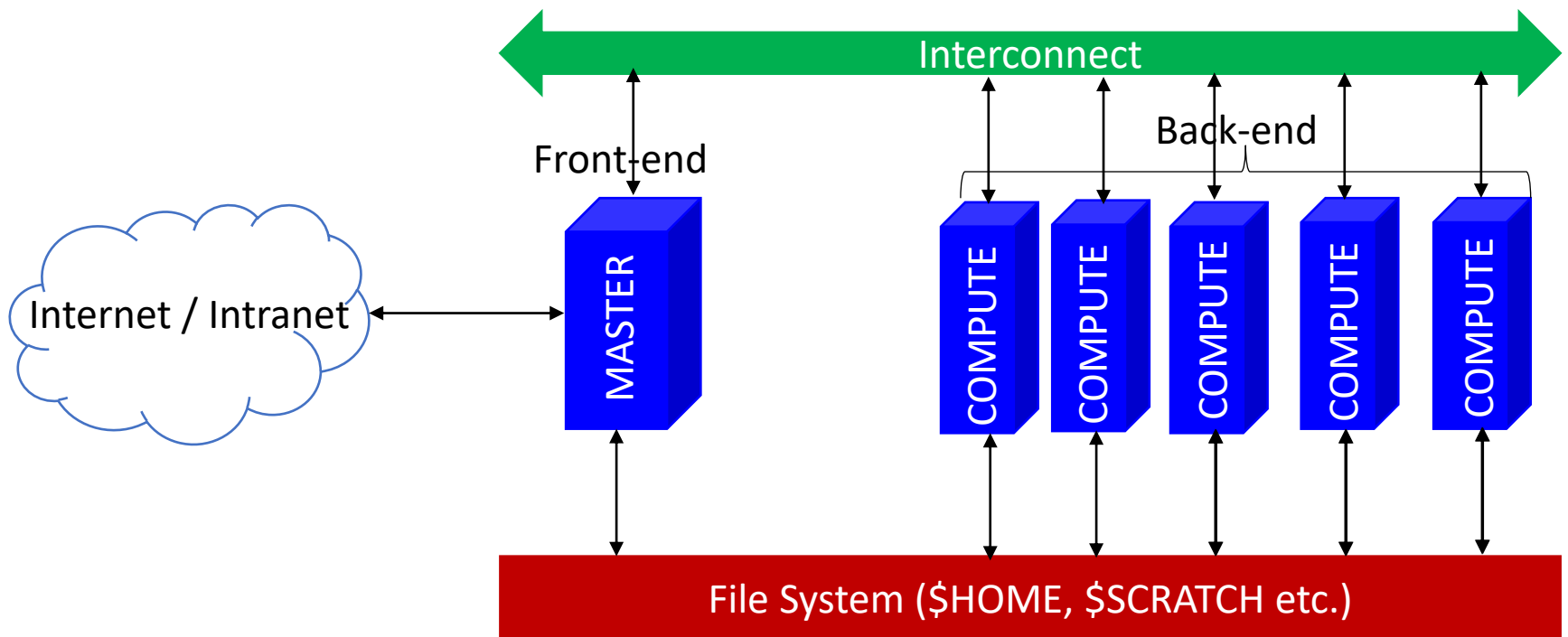
- Partitions/Queues
 - A logical grouping of (a subset of) nodes in the cluster
 - Single node can belong to multiple partitions (not done in practice)
 - Define attributes (and limits) for a job submitted to a Q
- E.g.
- A job submitted to a specific partition runs with low priority
 - A job can request a maximum of 4 cores
 - etc.

Cluster Elements - Software

- Operating System (OS), Software Development Tools, Applications
 - Linux-based OS in 100% of the supercomputing clusters in top500.org (2015 onwards)
 - Compiler tool chains, Runtime systems, Profilers
 - E.g. GCC, ICC, MPICH, JDK, Docker, Matlab, Intel Parallel Studio, NVProf, Tau etc.

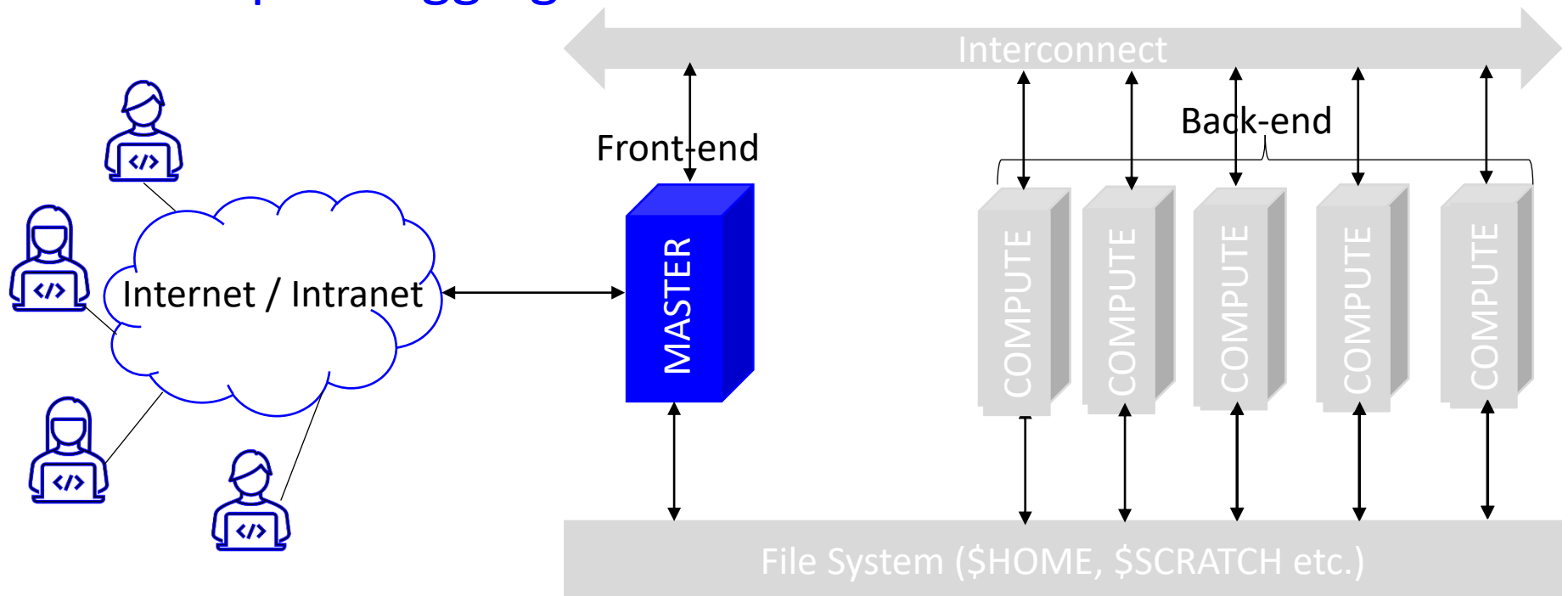
Recap

- Clusters



Clusters

- Step 1: Logging-in



Logging-in

- Logging into remote Linux system (master node) requires you to use SSH (“Secure Shell Protocol”)
 - Login credentials are encrypted
 - **SSH server** must be running on the system that you are logging into; Happens on most Linux systems by default.
 - **SSH client**, another piece of software, is used to authenticate and connect to the SSH server
 - Client software is available for all platforms (OSs)

Logging-in Windows

- Powershell on Windows 10
 - Press (Windows + R) -> Type “powershell”
 - Type “ssh <username>@<masternode_IP_address>”
 - Type ‘Yes’ when prompted (only first time)
 - Provide log-in credentials

 Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\ndheg> ssh nikhilh@10.250.101.100_
```

Logging-in Windows

- PuTTY SSH client Windows

- Download PuTTY from

<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html> (64-bit .exe)

- Double click on the

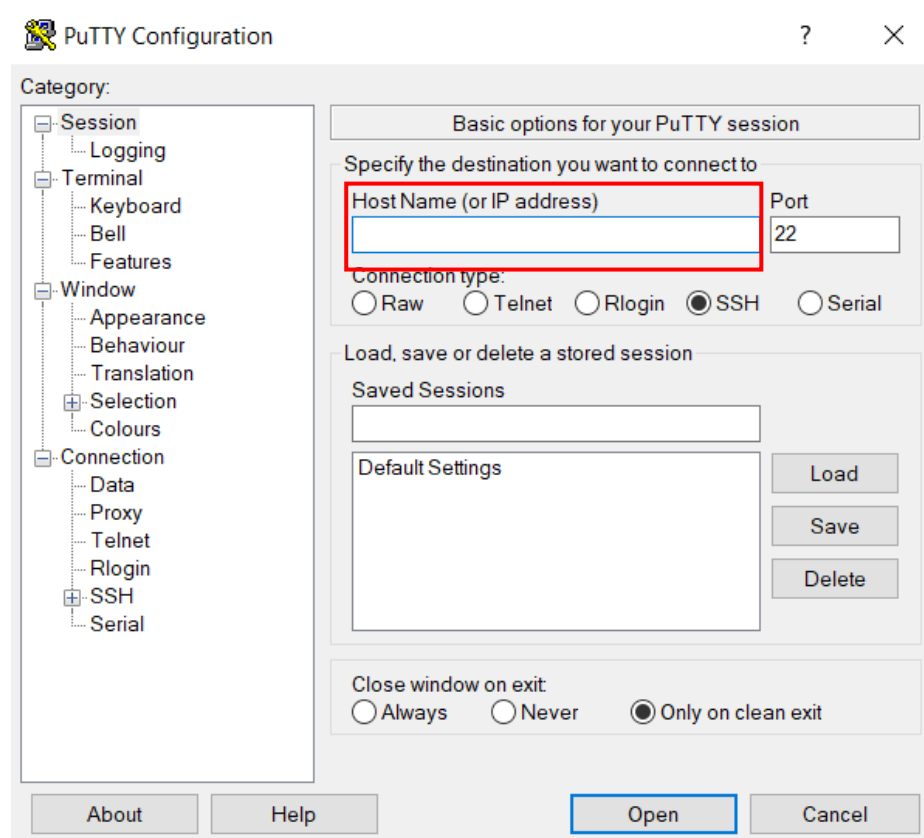


putty

icon after downloading

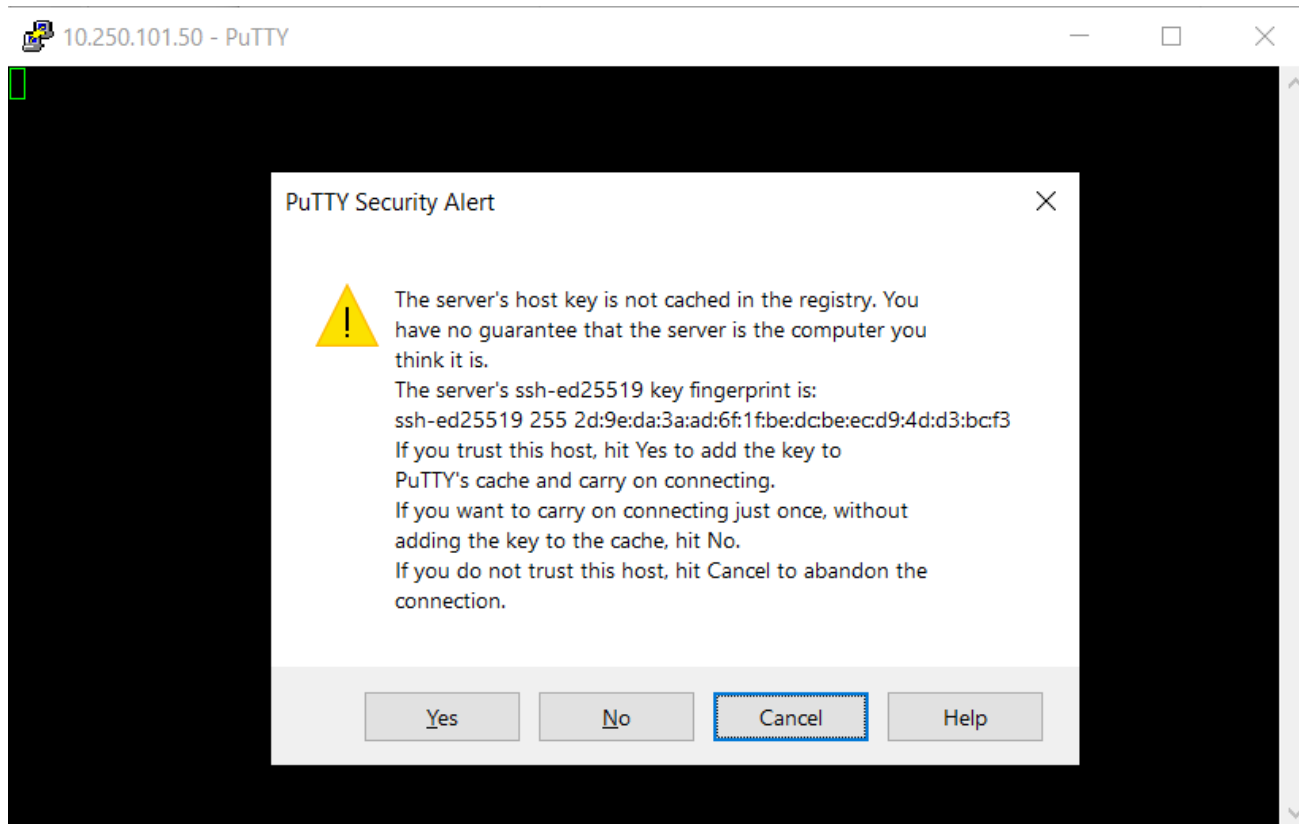
Logging-in Windows

- Type in the Host Name / IP address and click 'Open'



Logging-in Windows

- Click on 'Yes' (you are accepting the server host key)



Logging-in Windows

- Enter log-in credentials

 nikhilh@iitdhmaster:~

```
 login as: nikhilh
 nikhilh@10.250.101.100's password:
Last login: Wed Mar 17 09:53:33 2021 from 10.196.7.237
Intel(R) Parallel Studio XE 2020 Update 2 for Linux*
Copyright 2009-2020 Intel Corporation.
[nikhilh@iitdhmaster ~]$
```

Logging-in MAC

- Open the 'Terminal' program on MAC (Go -> Applications -> Terminal)

```
Last login: Sun Mar  7 11:35:13 on ttys000
```

```
The default interactive shell is now zsh.
```

```
To update your account to use zsh, please run `chsh -s /bin/zsh`.
```

```
For more details, please visit https://support.apple.com/kb/HT208050.
```

```
apples-MacBook-Pro:~ apple$ █
```

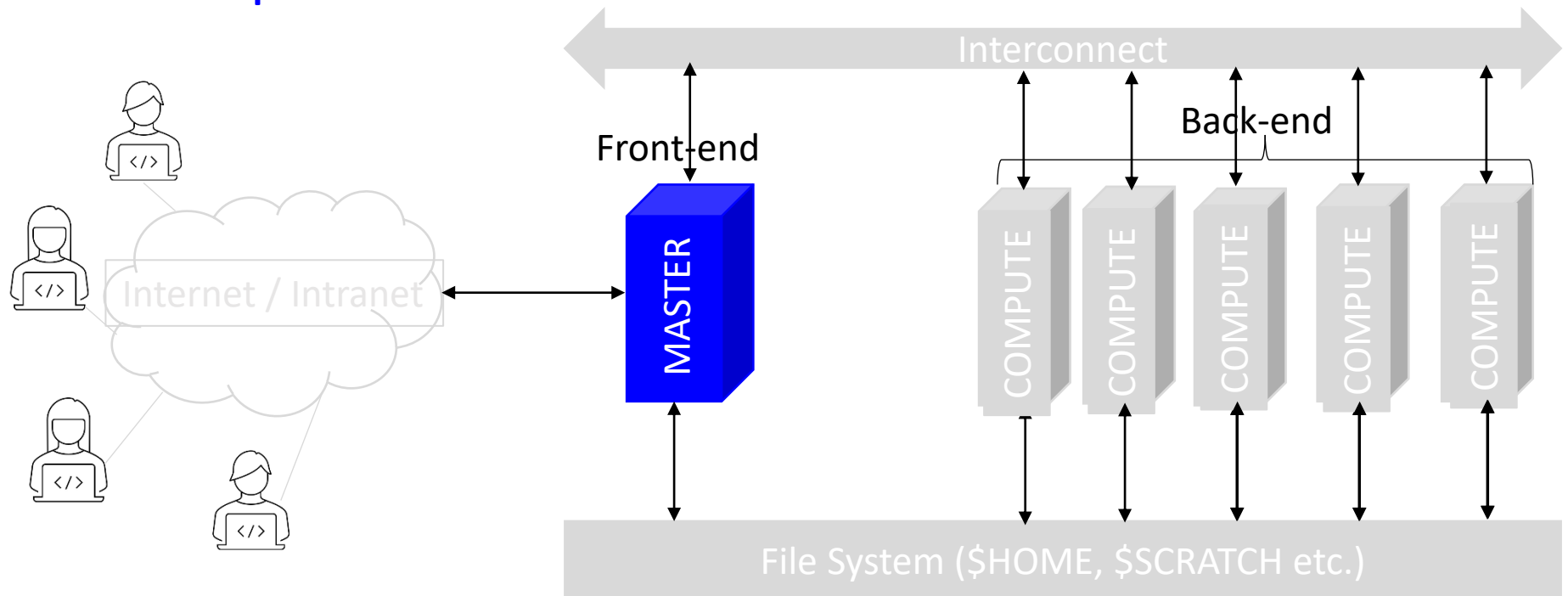
- Type “ssh <username>@<masternode_IP_address>
- Type 'Yes' when prompted (only first time)
- Provide log-in credentials

Logging-in Linux

- If you are a Linux user, you know what a 'Terminal' is 😊
- Type “ssh <username>@<masternode_IP_address>
- Type 'Yes' when prompted (only first time)
- Provide log-in credentials

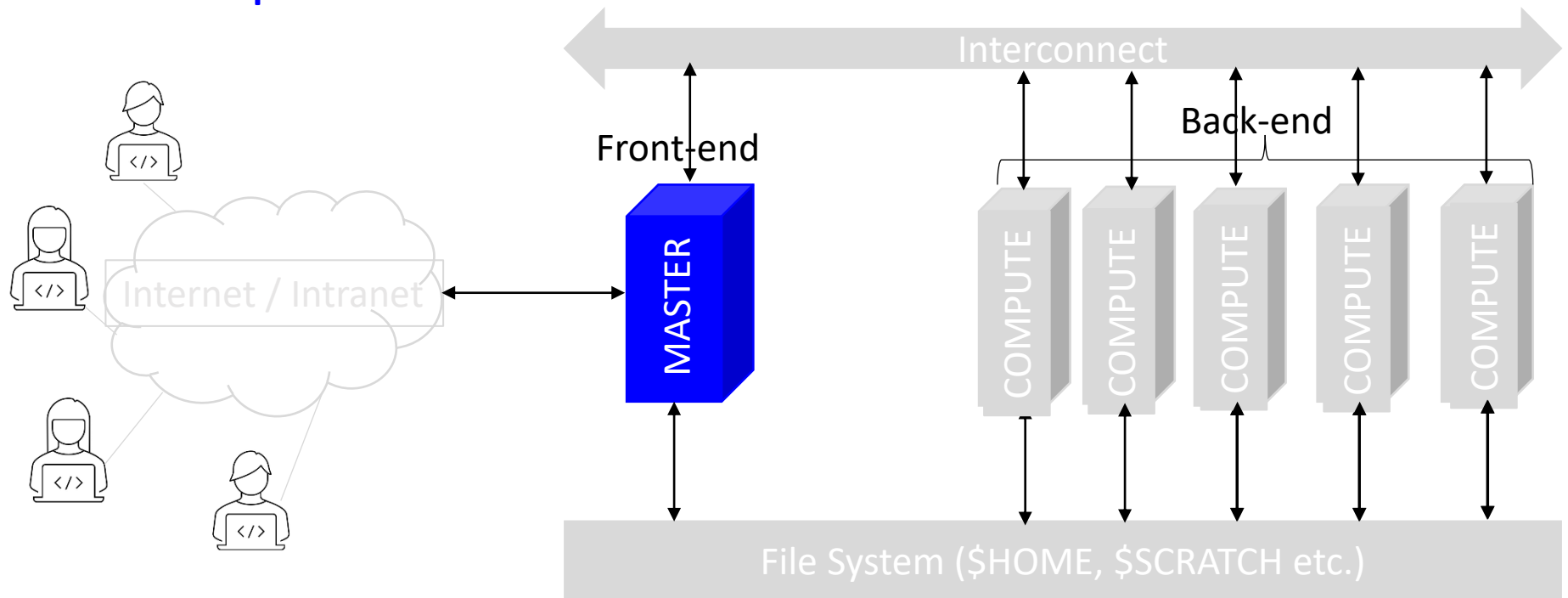
Clusters

- Step 2: Activities on the Master Node



Clusters

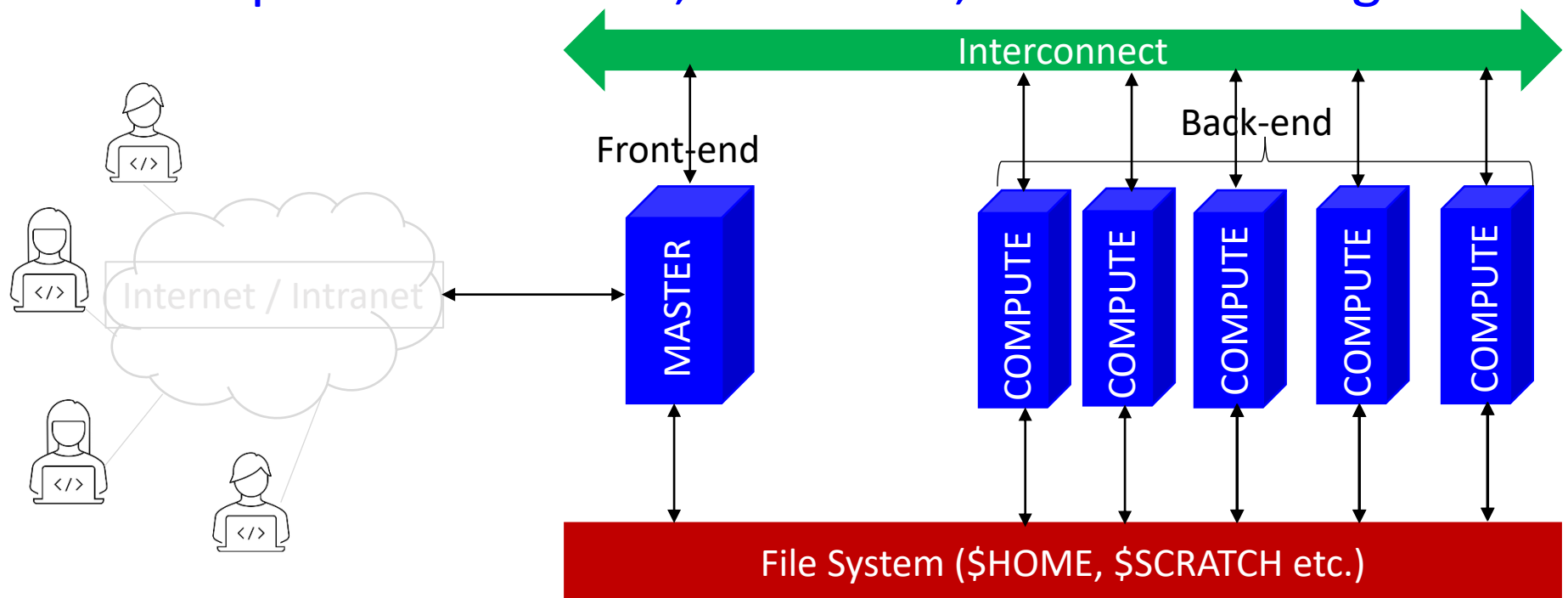
- Step 2: Activities on the Master Node



Master node runs Linux-OS. Let's review useful Linux commands

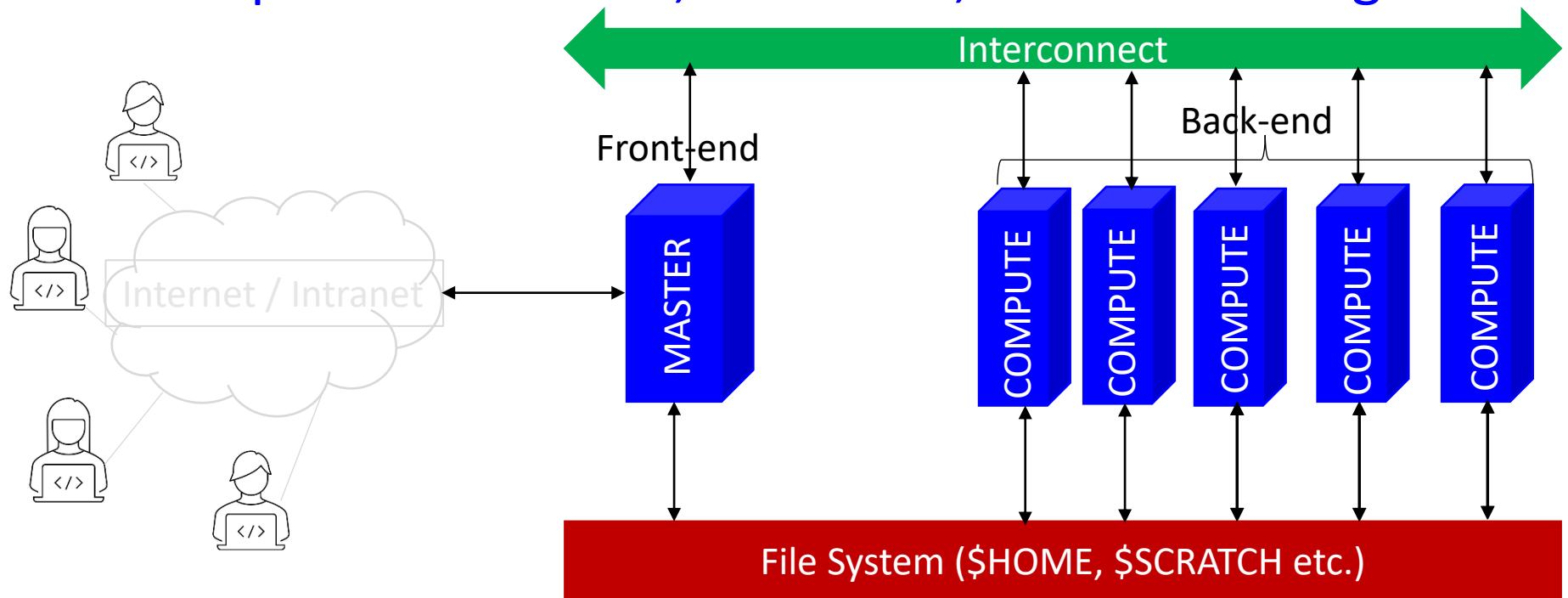
Clusters

- Step 3: Job creation, execution, and monitoring



Clusters

- Step 3: Job creation, execution, and monitoring



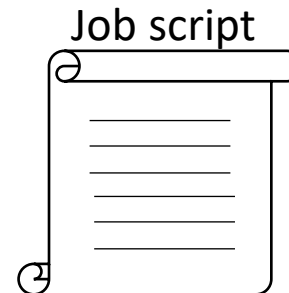
*User must go through the **Batch System** for this step*

Life of a Job

- Four Phases
 1. Creation
 2. Submission
 3. Execution
 4. Completion

Life of a Job - Creation

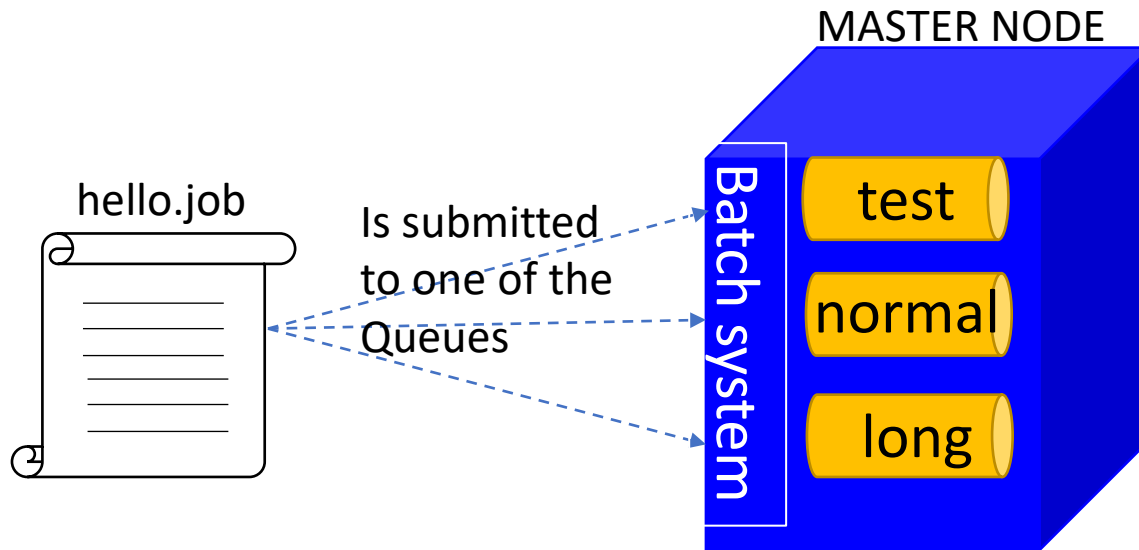
- Job creation specifies:
 1. What resources are needed (at least the following)
 - Maximum number of **nodes** and **cores** needed
 - Maximum amount of **time** needed
 - Maximum amount of memory needed
 - Whether exclusive access to a node is needed i.e. jobs from other users should run on the node simultaneously or not.
 - The partition or **queue** to run the job in
 2. What commands will be run in the job (actual task)
 - ./a.out
 - mpirun -np 10 helloworld
 - etc.
 3. **Output:** Job script



Life of a Job - Submission

- Successful submission implies:
 - No syntax errors in the specification
 - Amount of resources requested could possibly be allocated
 - A job object is created in the queue in which it would be executed
 - A job ID is returned to the user for reference
- Once the job is submitted (and before it is executed), you can change resource requirements (part 1) but not the actual task (part 2)

Life of a Job - Submission



- E.g. command: `sbatch hello.job`
- `test`, `normal`, and `long` are the names of queues/partitions on my cluster

Queues / Partition

- E.g. command: `sinfo`

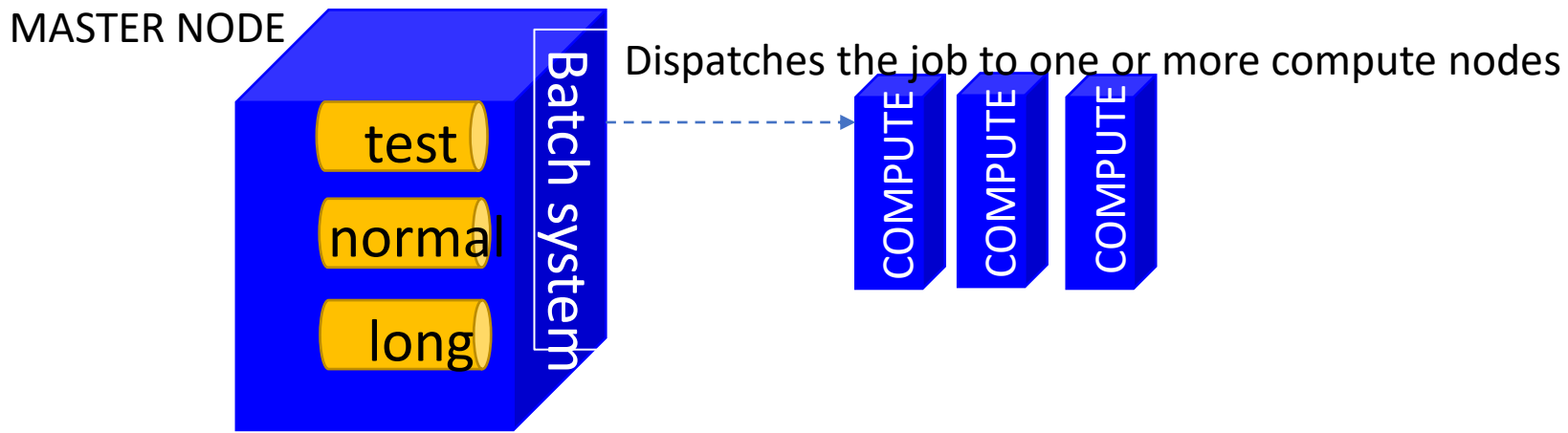
```
[nikhilh@iitdhmaster ~]$ sinfo
```

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
test	up	30:00	1	idle	cn01
long	up	6-00:00:00	4	idle	cn[02-05]
normal*	up	12:00:00	26	idle	cn[06-31]
gpu	up	12:00:00	1	idle	dgx

```
[nikhilh@iitdhmaster ~]$
```

Life of a Job - Execution

- When requested resources become available
 - Job is launched on the compute nodes
 - The commands specified in part 2 are executed in sequence
- Neither the job resources nor the actual task can be changed at this time. If job exceeds requested resources, it is terminated and a notification is sent.



Job Monitoring

- check the status (squeue)

```
[nikhilh@iitdhmaster hpc101]$ squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
851	normal	sh	nikhilh	R	1:08	1	cn06

```
[nikhilh@iitdhmaster hpc101]$
```

- delete the job if required (scancel)

```
> nikhilh@iitdhmaster:/iitdh/faculty/nikhilh/hpc101
```

```
[nikhilh@iitdhmaster hpc101]$ squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
851	normal	sh	nikhilh	R	1:08	1	cn06

```
[nikhilh@iitdhmaster hpc101]$ salloc -N 1 -p normal bash
```

```
salloc: Granted job allocation 852
```

```
[nikhilh@iitdhmaster hpc101]$ squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
852	normal	bash	nikhilh	R	0:07	1	cn06

```
[nikhilh@iitdhmaster hpc101]$ scancel 852
```

```
salloc: Job allocation 852 has been revoked.
```

```
[nikhilh@iitdhmaster hpc101]$ squeue
```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
-------	-----------	------	------	----	------	-------	------------------

```
[nikhilh@iitdhmaster hpc101]$
```

Life of a Job - Completion

- Can happen because of normal completion of execution, abnormal termination, or job exceeding requested resources
- Output (stdout/err) stored in file
 - At the same location where job was submitted (default)
 - At a location mentioned in the job script
- All resources are reclaimed at this point and allocated to other jobs if needed

Life of a Job - Completion

- job exceeding requested time

```
nikhilh@iitdhmaster:/iitdh/faculty/nikhilh/hpc101
[nikhilh@iitdhmaster hpc101]$ salloc -N 2 -t 00:01:00 -p normal bash
salloc: Granted job allocation 854
[nikhilh@iitdhmaster hpc101]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
test      up       30:00      1   idle cn01
long      up    6-00:00:00      4   idle cn[02-05]
normal*   up    12:00:00      2   mix  cn[06-07]
normal*   up    12:00:00     24   idle cn[08-31]
gpu       up    12:00:00      1   idle dgx
[nikhilh@iitdhmaster hpc101]$ squeue
JOBID PARTITION NAME      USER ST        TIME  NODES NODELIST(REASON)
854    normal  bash     nikhilh R          0:10      2  cn[06-07]
[nikhilh@iitdhmaster hpc101]$ salloc: Job 854 has exceeded its time limit and its allocation has been revoked.
```

- `mix` indicates non-exclusive access
- two nodes (indicated by `-N 2`) `cn06` and `cn07` are requested

Life of a Job - Completion

- job completing normally

```
[nikhilh@iitdhmaster hpc101]$ sbatch mwttest_large.job
Submitted batch job 855
[nikhilh@iitdhmaster hpc101]$ squeue
      JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
       855    normal mwttest_  nikhilh  R        0:03      8 cn[06-13]
[nikhilh@iitdhmaster hpc101]$ ls
hello.job  mwttest_large.job  slurm-844.out  slurm-845.out  slurm-855.out
```

- slurm-855.out contains the output of the program written to terminal (stdout)
 - In case of error you would have seen a file mwttest_large.job.e855 and mwttest_large.job.o855

Life of a Job - Completion

- Checking the status of completed job (qstat in PBS)

```
[nikhilh@iitdhmaster hpc101]$ qstat -u nikhilh  
iitdhmaster.iitdh.ac.in:  


| Job id | Username | Queue  | Name               | SessID | NDS | TSK | Req'd<br>Memory | Req'd<br>Time | Elap<br>Use S | Time  |
|--------|----------|--------|--------------------|--------|-----|-----|-----------------|---------------|---------------|-------|
| 855    | nikhilh  | normal | mwtttest_large.job | --     | 8   | 40  | --              | 00:30         | C             | 00:00 |


```

- In SLURM (sacct)

```
[nikhilh@iitdhmaster hpc101]$ sacct -j 855  


| JobID     | JobName     | Partition | Account | AllocCPUS | State     | ExitCode |
|-----------|-------------|-----------|---------|-----------|-----------|----------|
| 855       | mwtttest_l+ | normal    | root    | 40        | COMPLETED | 0:0      |
| 855.batch | batch       |           | root    | 5         | COMPLETED | 0:0      |
| 855.0     | hydra_bst+  |           | root    | 8         | COMPLETED | 0:0      |

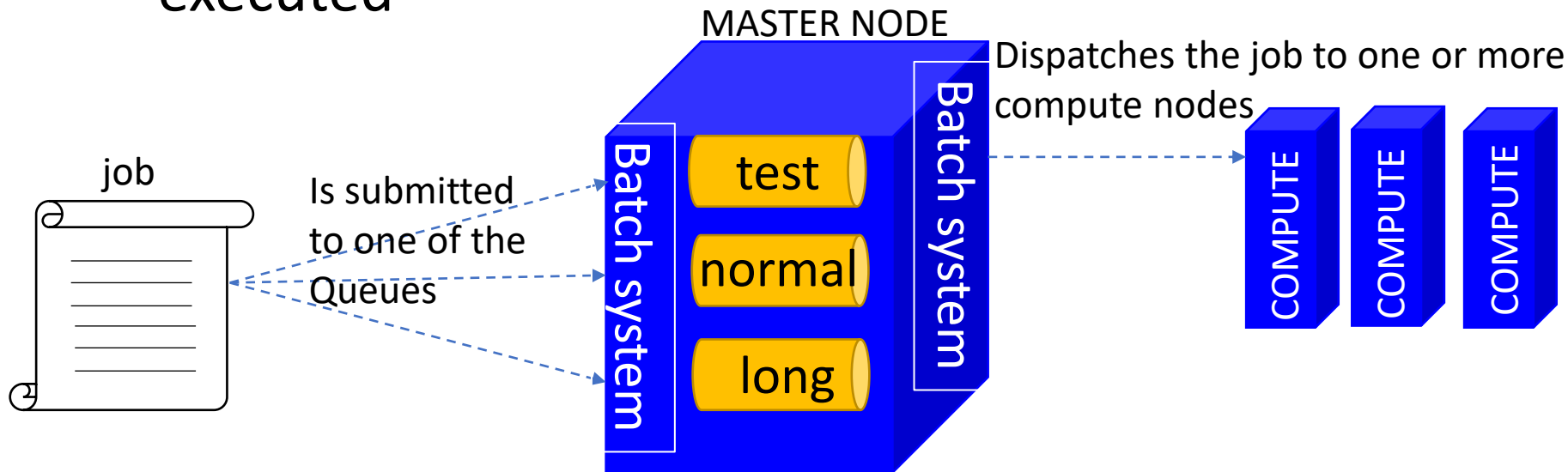
  
[nikhilh@iitdhmaster hpc101]$
```

Recap - SLURM Commands

- `sbatch` → to submit a job to the queue for later execution (batch mode)
- `scancel` → to cancel a queued or running job
- `sinfo` → report system info (queues, nodes, etc.)
- `squeue` → report job status
- `srun` → to create a job and execute
- `sattach` → to connect stdin/out/err to an existing job
- `sreport` → report resource usage by user, account etc.
- `salloc` → to create a job and start a shell to use it (interactive mode)
- `sstat` → report accounting info about currently running jobs

Batch System (SLURM and PBS)

- Runs continuously (daemon) and monitors resources
- Scans the queues repeatedly
 - user jobs are submitted to queues
- Determines when and where the jobs are to be executed



SLURM and PBS

- Provide exclusive or non-exclusive access to compute nodes
- Provide a set of user-commands for
 - job control
 - node control
 - accounting
 - Many other things ...

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node 5
#echo commands to stdout
#set -x

#run MPI program
mpirun -np 40 /iitdh/faculty/nikhilh/Nikhil_test/exes/MWT
```

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmast  
#!/bin/bash  
#SBATCH -p normal  
#SBATCH -t 00:30:00  
#SBATCH -N 8  
#SBATCH --ntasks-p  
#echo commands to  
#set -x  
  
#run MPI program  
mpirun -np 40 /iitdh/faculty/nikhilh/Nikhil_test/exes/MWT
```

First line. Always looks like this: first two characters are # and !. This line means that the job would be starting in job owner's login shell environment. In this case the shell is 'bash'

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node 5
#echo commands to stdout
#set -x

#run MPI program
mpirun -np 40 /iitdh/faculty/nikhilh/Nikhil_test/exes/MWT
```

Means that the partition (p) to be used for placing the job is 'normal'

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
```

```
#!/bin/bash
```

```
#SBATCH -p normal
```

```
#SBATCH -t 00:30:00
```

```
#SBATCH -N 8
```

```
#SBATCH --ntasks-per-node 5
```

```
#echo commands to stdout
```

```
#set -x
```

```
#run MPI program
```

```
mpirun -np 40 /iitdh/faculty/nikhilh/Nikhil_test/exes/MWT
```

Means that the maximum time this job could take is 30 mins (note -t option)

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node=1
# echo commands to stdout
# set -x

#run MPI program
mpirun -np 40 /iitdh/faculty/nikhilh/Nikhil_test/exes/MWT
```

Means that the 8 nodes are needed by this job (note -N option)

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node 5
#echo commands to stdout
#set -x

#run MPI program
mpirun -np 40 /iitdh/faculty/n
```

Means that the 5 cores on each of the nodes requested are needed. Remaining cores on the nodes can be given away to other jobs (because of non-exclusive access)

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node 5
#echo commands to stdout
#set -x

#run MPI program
mpirun -np 40 /iitdh/faculty/
```

Any line beginning with a # is a 'comment' in a shell program. Previous lines have #SBATCH, which is interpreted by the batch system SLURM

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node 5
#echo comm
#set -x ← sets a flag on shell program. All
#run MPI p executed commands are printed
mpirun -np 40 /iitdh/faculty/nikhiln/nikhil_test/exes/MWT
```

Creating Job Scripts

- Still haven't looked at inside of a .job file!

```
[nikhilh@iitdhmaster hpc101]$ cat mwttest_large.job
#!/bin/bash
#SBATCH -p normal
#SBATCH -t 00:30:00
#SBATCH -N 8
#SBATCH --ntasks-per-node
#echo commands to stdout
#set -x

#run MPI program
mpirun -np 40 /iitdh/faculty/nikhilh/Nikhil_test/exes/MWT
```

Part 2 of the specification.
Command to perform the actual task. mpirun is the program that launches 40 copies of the executable MWT

SLURM/PBS Environment Variables

- `$SLURM_JOB_ID`
- `$SLURM_JOB_NAME`
- `$SLURM_SUBMIT_DIR`
- `$SLURM_NTASKS`
- `$SLURM_NTASKS_PER_NODE`
- `$SLURM_JOB_NUM_NODES`
- `$PBS_NODEFILE`
 - (in SLURM you will have to do `srun hostname`)